Induction and Recursion

5

Chapter Summary

- Mathematical Induction
- Strong Induction
- Well-Ordering
- Recursive Definitions
- Structural Induction
- Recursive Algorithms
- Program Correctness

5.1 Mathematical Induction

- Mathematical Induction is used to prove propositions of the form $\forall n P(n)$, where the universe of discourse is the set of positive integers.

- It is extremely important to note that mathematical induction can be used only to prove results obtained in some other way. It is not a tool for discovering formulae or theorems.

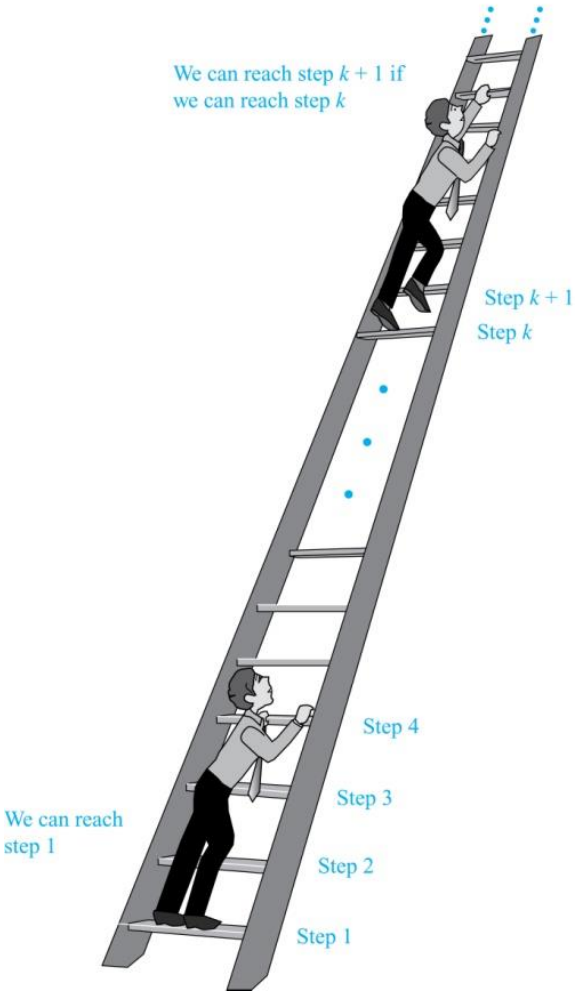
Climbing an Infinite Ladder

Suppose we have an infinite ladder:

1. We can reach the first rung of the ladder.
2. If we can reach a particular rung of the ladder, then we can reach the next rung.

From (1), we can reach the first rung. Then by applying (2), we can reach the second rung. Applying (2) again, the third rung. And so on. We can apply (2) any number of times to reach any particular rung, no matter how high up.

This example motivates proof by mathematical induction.



Basic and Inductive Steps

- A proof by mathematical induction that $P(n)$ is true for every positive integer n consists of two steps:
  - *Basis step*: The proposition $P(1)$ is shown to be true.
  - *Inductive step*: The implication $P(n) \rightarrow P(n+1)$ is shown to be true for every positive integer n.
- Expressed as a rule of inference, this proof technique can be stated as

$$[P(1) \wedge \forall n(P(n) \rightarrow P(n+1))] \rightarrow \forall n P(n)$$

In a proof by mathematical induction it is not assumed that $P(n)$ is true for all positive integers!

It is only shown that if it is assumed that $P(n)$ is true, then $P(n+1)$ is also true.

Thus, a proof by mathematical induction is not a case of circular reasoning.

Validity of Mathematical Induction

- Mathematical induction is valid because of the **well ordering property**, which states that every nonempty subset of the set of positive integers has a least element (*see Section* 5.2 *and Appendix* 1). Here is the proof:
  - Suppose that $P(1)$ holds and $P(k) \rightarrow P(k+1)$ is true for all positive integers k.
  - Assume there is at least one positive integer n for which $P(n)$ is false. Then the set S of positive integers for which $P(n)$ is false is nonempty.
  - By the well-ordering property, S has a least element, say m.
  - We know that m can not be 1 since $P(1)$ holds.
  - Since m is positive and greater than 1, $m - 1$ must be a positive integer. Since $m - 1 < m$, it is not in S, so $P(m - 1)$ must be true.
  - But then, since the conditional $P(k) \rightarrow P(k+1)$ for every positive integer k holds, $P(m)$ must also be true. This contradicts $P(m)$ being false.
  - Hence, $P(n)$ must be true for every positive integer n.

Examples

Example: Show that: $\sum_{i=1}^{n} = \frac{n(n+1)}{2}$

Solution:

- **BASIS STEP: *P*(1) is true since 1(1 + 1)/2 = 1.**
- **INDUCTIVE STEP: Assume true for *P*(*k*).**

The inductive hypothesis is $\sum_{i=1}^{k} = \frac{k(k+1)}{2}$

Under this assumption,

$$1+2+\ldots+k+(k+1) = \frac{k(k+1)}{2} + (k+1)$$
$$= \frac{k(k+1)+2(k+1)}{2}$$
$$= \frac{(k+1)(k+2)}{2}$$

Examples

Example: Use mathematical induction to prove that

$$n < 2^n \text{ for all positive integers } n.$$

Solution: Let $P(n)$ be the proposition that $n < 2^n$.

- **BASIS STEP: $P(1)$ is true since $1 < 2^1 = 2$.**
- **INDUCTIVE STEP: Assume $P(k)$ holds, i.e., $k < 2^k$, for an arbitrary positive integer k.**
- **Must show that $P(k + 1)$ holds. Since by the inductive hypothesis, $k < 2^k$, it follows that:**

$$k + 1 < 2^k + 1 \leq 2^k + 2^k = 2 \cdot 2^k = 2^{k+1}$$

Therefore $n < 2^n$ holds for all positive integers n.

Examples

Example: Use mathematical induction to show that if S is a finite set with n elements, where n is a nonnegative integer, then S has 2^n subsets.

(*Chapter 6 uses combinatorial methods to prove this result.*)

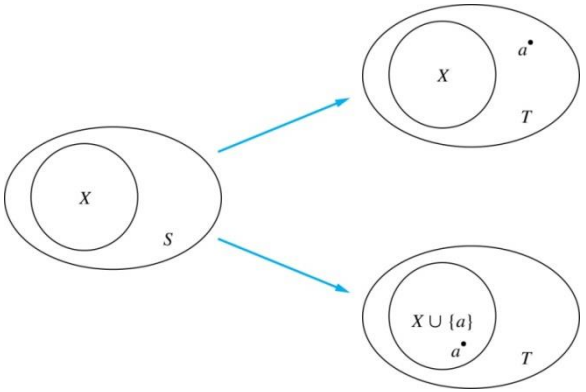
Solution: Let $P(n)$ be the proposition that a set with n elements has 2^n subsets.

- **Basis Step: $P(0)$ is true, because the empty set has only itself as a subset and $2^0 = 1$.**
- **Inductive Step: Assume $P(k)$ is true for an arbitrary nonnegative integer k.**
- **… …**

Continues

Inductive Hypothesis: For an arbitrary nonnegative integer k, every set with k elements has 2^k subsets.

- **Let T be a set with $k + 1$ elements. Then $T = S \cup \{a\}$, where $a \in T$ and $S = T - \{a\}$. Hence $|S| = k$.**
- **For each subset X of S, there are exactly two subsets of T, i.e., X and $X \cup \{a\}$.**



- **By the inductive hypothesis S has 2^k subsets. Since there are two subsets of T for each subset of S, the number of subsets of T is $2 \cdot 2^k = 2^{k+1}$.**

Exercises

- **P350: 44, 61**

5.2 Strong Induction and Well -Ordering

- Basic step: The proposition $P(1)$ is shown to be true.
- Inductive step: It is shown that
- $[P(1) \wedge P(2) \wedge \ldots \wedge P(n)] \rightarrow P(n+1)$
- is shown to be true for every positive integer .

- The two forms of mathematical induction are equivalent.

Examples

Example: Show that if n is an integer greater than 1, then n can be written as the product of primes.

Solution: Let $P(n)$ be the proposition that n can be written as a product of primes.

- **BASIS STEP: $P(2)$ is true since 2 itself is prime.**
- **INDUCTIVE STEP: The inductive hypothesis is $P(j)$ is true for all integers j with $2 \leq j \leq k$. To show that $P(k + 1)$ must be true under this assumption, two cases need to be considered:**
  - **If $k + 1$ is prime, then $P(k + 1)$ is true.**
  - **Otherwise, $k + 1$ is composite and can be written as the product of two positive integers a and b with $2 \leq a \leq b < k + 1$. By the inductive hypothesis a and b can be written as the product of primes and therefore $k + 1$ can also be written as the product of those primes.**

Hence, it has been shown that every integer greater than 1 can be written as the product of primes.

(*uniqueness ?*)

Using Strong Induction in Computational Geometry

- Self-study

Examples

Example: Prove that every amount of postage of 12 cents or more can be formed using just 4-cent and 5-cent stamps.

Solution: Let $P(n)$ be the proposition that postage of n cents can be formed using 4-cent and 5-cent stamps.

- **BASIS STEP: $P(12)$, $P(13)$, $P(14)$, and $P(15)$ hold.**
  - **$P(12)$ uses three 4-cent stamps.**
  - **$P(13)$ uses two 4-cent stamps and one 5-cent stamp.**
  - **$P(14)$ uses one 4-cent stamp and two 5-cent stamps.**
  - **$P(15)$ uses three 5-cent stamps.**
- **INDUCTIVE STEP: The inductive hypothesis states that $P(j)$ holds for $12 \leq j \leq k$, where $k \geq 15$. Assuming the inductive hypothesis, it can be shown that $P(k + 1)$ holds.**
- **Using the inductive hypothesis, $P(k - 3)$ holds since $k - 3 \geq 12$. To form postage of $k + 1$ cents, add a 4-cent stamp to the postage for $k - 3$ cents.**

Hence, $P(n)$ holds for all $n \geq 12$

The well-ordering property

- The validity of mathematical induction follows from the following fundamental axiom about the set of integers:

  The Well-Ordering Property: Every nonempty set of nonnegative integers has a least element.

- The well-ordering property can be generalized.
  - Definition: A set is *well ordered if every subset has a least element.*
    - N is well ordered under ≤.
    - The set of finite strings over an alphabet using lexicographic ordering is well ordered.
  - We will see a generalization of induction to sets other than the integers in the next section.

Well-Ordering Property

Example: Use the well-ordering property to prove the division algorithm, which states that if a is an integer and d is a positive integer, then there are unique integers q and r with $0 \leq r < d$, such that $a = dq + r$.

Solution: Let S be the set of nonnegative integers of the form $a - dq$, where q is an integer. The set is nonempty since $-dq$ can be made as large as needed.

- **By the well-ordering property, S has a least element $r = a - dq_0$. The integer r is nonnegative. It also must be the case that $r < d$. If it were not, then there would be a smaller nonnegative element in S, namely, $a - d(q_0 + 1) = a - dq_0 - d = r - d > 0$.**
- **Therefore, there are integers q and r with $0 \leq r < d$.**

(*uniqueness of q and r is Exercise* 37)

Example

- In a round-robin tournament every player plays every other player exactly once and each match has a winner and loser. We say that the players $p1$, $p2$, …, p_m form a cycle if $p1$ beats $p2$, $p2$ beats $p3$, …, p_m-1 beats p_m, and p_m beats $p1$. Using the well-ordering principle to show that if there is a cycle of length m ($m \geq 3$) among the players in a round-robin tournament, there must be a cycle of three of these player.

Solution

- Assume that there is no cycle of three players. Since there is at least one cycle in the round-robin tournament, the set of all positive integers n for which there is a cycle of length n is nonempty.
- By the well-ordering property, this set of positive integers has a least element k, which by assumption must be greater than 3. Consequently, there exists a cycle of players $p1$, $p2$, $p3$, …, p_k and no shorter cycle exists.
- Consider the first three elements of this cycle, $p1$, $p2$, $p3$,. There are two possible outcomes of the match between $p1$ and $p3$.

Infinite descent

- Used to show that for a propositional function $p(n)$, **$p(k)$ is false for all positive integers k**.
- If $p(k)$ is true for at least one integer k, then the well-ordering property implies that there is a least positive integer s such that $p(s)$ is true.
- The method proceeds by finding a positive integer s' with $s'<s$ for which the propositional function is true. It follows that $p(n)$ must be false for all positive integers.
- This technique is called the method of infinite decent since the procedure of finding smaller integers for which the propositional function is true could be continued indefinitely.

Example

- Show $\sqrt{2}$ is irrational using infinitely descent.

Solution: $\sqrt{2} = m/n \Rightarrow \sqrt{2} = M/N$

(N is a least integer for some M)

To carry out proof by infinite descent, show

$\sqrt{2} = (2N-M)/(M-N)$ and $0<M-N<N$

Only need to show

$(2N-M)/(M-N) = M/N$

Use $(M/N)^2=2$

$(2N-M)/(M-N) = M/N$

Why mathematical induction is valid

- Suppose we know that $P(1)$ is true and that the proposition $P(n) \rightarrow P(n+1)$ is true for all positive integers n.
- To show that $P(n)$ must be true for all positive integers, assume that there is at least one positive integer for which $P(n)$ is false.
- Then the set S of positive integers for which $P(n)$ is false is nonempty.
- By the well-ordering property, S has a least element, which will be denoted by k. We know that k cannot be 1, since $P(1)$ is true.
- Since $k > 1$, $k-1$ is a positive integer.
- Furthermore, since $k-1$ is less than k, it is not in S, so $P(k-1)$ must be true.
- Since the implication $P(k-1) \rightarrow P(k)$ is also true, it must be the case that $P(k)$ is true.
- This contradicts the choice of k.
- Hence, $P(n)$ must be true for every positive integer n.

Exercises

- P.363: 12, 25, 31, 37

5.3 Recursive Definitions and Structural Induction

- Recursively Defined Functions
- Recursively Defined Sets and Structures
- Structural Induction
- Generalized Induction

Recursively defined functions

- Recursive/inductive definition: define a function with the set of nonnegative integers as its domain:
- Basic step: Specify the value of the function at zero.
- Recursive step: Give a rule for finding its value as integer from its values at small integers.
- In some recursive definitions of functions, the values of the function at the first *k* positive integers are specified, and a rule is given for determining the value of the function at larger integers from its values at some or all of the proceeding *k* integers. (strong mathematical induction).

Examples [1]

- Suppose that f is defined recursively by

  $f(0)=3$

  $f(n+1) = 2f(n)+3$

  Find $f(1)$, $f(2)$, and $f(3)$

  $f(1) = 9$, $f(2) = 21$, $f(3)=45$

- Give a recursive definition of the factorial function $F(n) = n!$.

  $F(0) = 1$

  $F(n+1) = (n+1)F(n)$

Recursively defined functions are well-defined

Let $P(n)$ be the statement "f is well-defined at n .

(1) $P(0)$ is true.

(2) Assume that $P(n)$ is true. Then f is well-defined at $n+1$.

Since $f(n+1)$ is given in terms of $f(n)$.

Fibonacci numbers

The Fibonacci numbers can be defined

$$f_0 = 0$$

$$f_1 = 1$$

$$f_n = f_{n-1} + f_{n-2} \text{ for } n=2,3,4,\ldots$$

Show that $f_n > \alpha^{n-2}$ where $\alpha = \frac{1+\sqrt{5}}{2}$ whenever $n \geq 3$.

Proof:

(1) Inductive base

$$f_3 = 2 > \alpha = \frac{1+\sqrt{5}}{2} \qquad f_4 = 3 > \alpha^2 = (\frac{1+\sqrt{5}}{2})^2$$

(2) Inductive step

Assume that *P*(*k*) is true, namely, that $f_k > \alpha^{k-2}$ for all integers *k* with $3 \leq k \leq n$.

We must show that *P*(*k*+1) is true, that is $f_{k+1} > \alpha^{k-1}$.

Since α is the solution of $x^2 - x - 1 = 0$, it follows that $\alpha^2 = \alpha + 1$

Therefore,

$$\alpha^{k-1} = \alpha^2 \cdot \alpha^{k-3} = (\alpha+1) \cdot \alpha^{k-3} = \alpha^{k-2} + \alpha^{k-3}$$

By the inductive hypothesis, if $n \geq 5$, it follows that

$$f_{k-1} > \alpha^{k-3}, f_k > \alpha^{k-2}$$

Therefore, $f_{k+1} = f_{k-1} + f_k > \alpha^{k-3} + \alpha^{k-2} = \alpha^{k-1}$

Euclidean algorithm

Find the greatest common divisor of the positive integers *a* and *b*, where *a*≥*b*.

Let $a=bq+r$, where a, b, q, and r are integers.

Then gcd (a, b)=gcd (r, b)

$$a = r_0 \,, b = r_1$$

$$r_0 = r_1 q_1 + r_2 \qquad 0 \le r_2 < r_1$$

$$r_1 = r_2 q_2 + r_3 \qquad 0 \le r_3 < r_2$$

$$\dots$$

$$r_{n-2} = r_{n-1} q_{n-1} + r_n \qquad 0 \le r_n < r_{n-1}$$

$$r_{n-1} = r_n q_n$$

$$\gcd(a, b) = \gcd(r_{n-1}, r_n) = r_n$$

Example

For example, find gcd (414,662)

662=414×1+248

414=248×1+166

248=166×1+82

166=82×2+2

82=2×41

hence, gcd (414,662)=2

LAME'S Theorem

- Let a , b be positive integers with $a \geq b$. Then the number of divisions used by the Euclidean algorithm to find gcd (a, b) is less than or equal to five times the number of decimal digits in b.

Proof

$$r_0 = r_1 q_1 + r_2 \qquad 0 \le r_2 < r_1$$
$$r_1 = r_2 q_2 + r_3 \qquad 0 \le r_3 < r_2$$
$$\cdots$$
$$r_{n-2} = r_{n-1} q_{n-1} + r_n \qquad 0 \le r_n < r_{n-1}$$
$$r_{n-1} = r_n q_n$$

Proof:

$n \le 5k$

From the above equations,

$q_1, q_2, \cdots, q_{n-1} \ge 1$ $\qquad$ $q_n \ge 2 \quad \because r_n < r_{n-1}$

This implies

$$r_n \ge 1 = f_2$$
$$r_{n-1} \ge 2r_n \ge 2f_2 = f_3$$
$$r_{n-2} \ge r_{n-1} + r_n \ge f_3 + f_2 = f_4$$
$$r_{n-3} \ge r_{n-2} + r_{n-1} \ge f_3 + f_4 = f_5$$

$\cdots$

$$r_2 \geq r_3 + r_4 \geq f_{n-1} + f_{n-2} = f_n$$

$$b = r_1 \geq r_2 + r_3 \geq f_n + f_{n-1} = f_{n+1}$$

$$\because f_{n+1} > \alpha^{n-1} \text{ for } n > 2$$

$$\therefore b > \alpha^{n-1}$$

$$\log_{10} b > (n-1)\log_{10} \alpha > \frac{n-1}{5}$$

$$n - 1 < 5 \cdot \log_{10} b < 5k$$

$$\therefore n \leq 5k$$

Recursively Defined Sets and structures

Recursively Defined Sets

- Sets can be defined recursively.

  Basis Step: Specify an initial collection of elements.

  Recursive Step: Give the rules for constructing elements of the set from other elements already in the set.

Examples

- Consider the subset of the set of integers defined by

  Basis Step: $3 \in S$

  Recursive Step: if $x \in S$ and $y \in S$, then $x+y \in S$

- The set $\sum^*$ of stings over the alphabet $\sum$ can be defined recursively by

  Basis Step: $\lambda \in \sum^*$

  Recursive Step: if $w \in \sum^*$ and $x \in \sum$, then $wx \in \sum^*$

Examples

- Two strings can be combined via the operation of concatenation. Let $\sum$ be a set of symbols and $\sum^*$ the set of strings formed from symbols in $\sum$. We can defined concatenation of two strings, denoted by $\bullet$, recursively as follows.

  Basis Step: if $w \in \sum^*$, then $w \bullet \lambda = w$.

  Recursive Step: if $w_1 \in \sum^*$ and $w_2 \in \sum^*$ and $x \in \sum$,

  then $w_1 \bullet (w_2 x) = (w_1 \bullet w_2)x$.

- Length of a string: give a recursive definition of $l(w)$.

  Basis Step: $l(\lambda) = 0$.

  Recursive Step: $l(wx) = l(w)+1$ if $w \in \sum^*$ and $x \in \sum$

Well-formed formulae for compound proposition

- **We can define the set of well-formed formulae for compound propositions involve T,F, propositional variables, and operators from the set $\{\leftrightarrow, \rightarrow, \neg, \wedge, \vee\}$.**

  Basis Step: ***T*, *F*, and *p*, where *p* is a propositional variable, are well-formed formulae.**

  Recursive Step: **$(\neg p)$, $(p \vee q)$, $(p \wedge q)$, $(p \rightarrow q)$, $(p \leftrightarrow q)$ are well-formed formulae if *p* and *q* are well-formed formulae.**

Well-formed formulae of operator and operands

- **We can define the set of well-formed formulae consisting of variables, numerals, and operators from the set {+, -, *, /, ↑} recursively.**

  Basis Step: *x* is a well-formed formula if x is a numeral or variable.

  Recursive Step: If *F* and *G* are well-formed formulae, then (*F*+*G*), (*F*-*G*), (*F*G*), (*F*/*G*), and(*F*↑*G*) are well-formed formulae.**

Recursively define trees

- Rooted trees
- Extended binary trees
- Full binary trees

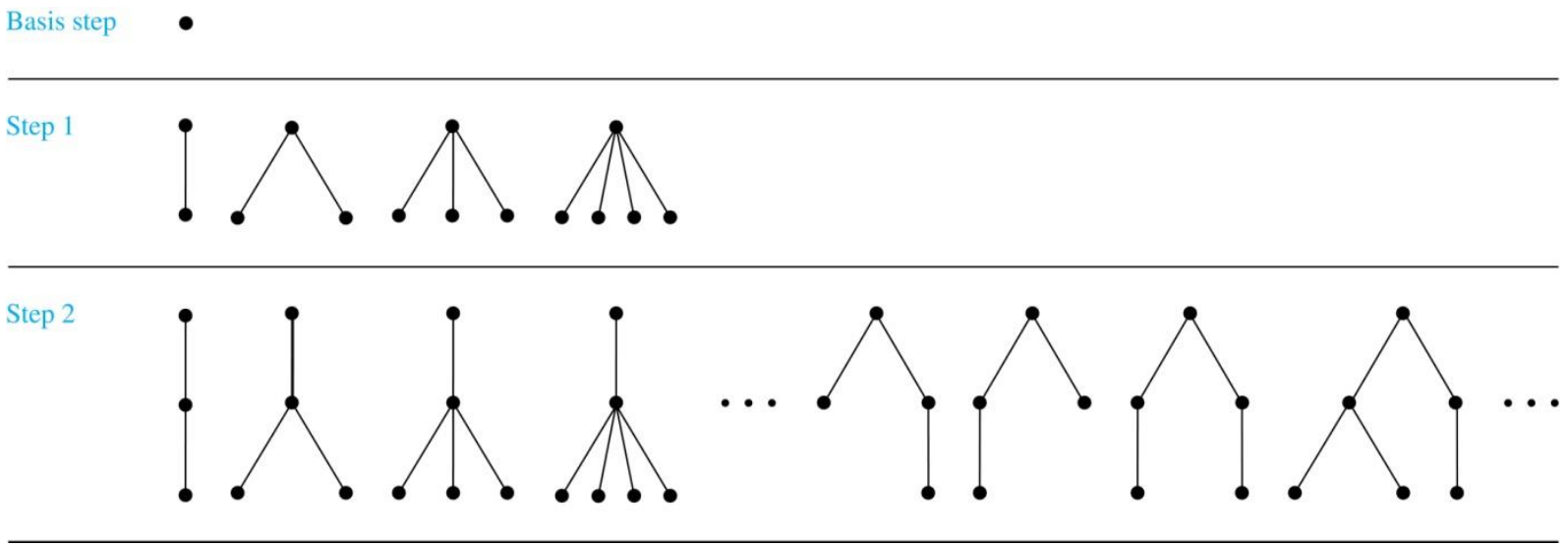
Rooted Trees

Definition: The set of *rooted trees*, where a rooted tree consists of a set of vertices containing a distinguished vertex called the *root*, and edges connecting these vertices, can be defined recursively by these steps:

BASIS STEP: A single vertex r is a rooted tree.

RECURSIVE STEP: Suppose that $T_1, T_2, \ldots, T_n$ are disjoint rooted trees with roots $r_1, r_2, \ldots, r_n$, respectively. Then the graph formed by starting with a root r, which is not in any of the rooted trees $T_1, T_2, \ldots, T_n$, and adding an edge from r to each of the vertices $r_1, r_2, \ldots, r_n$, is also a rooted tree.

Building Up Rooted Trees



- Trees are studied extensively in Chapter 11.
- Next we look at a special type of tree, the full binary tree.

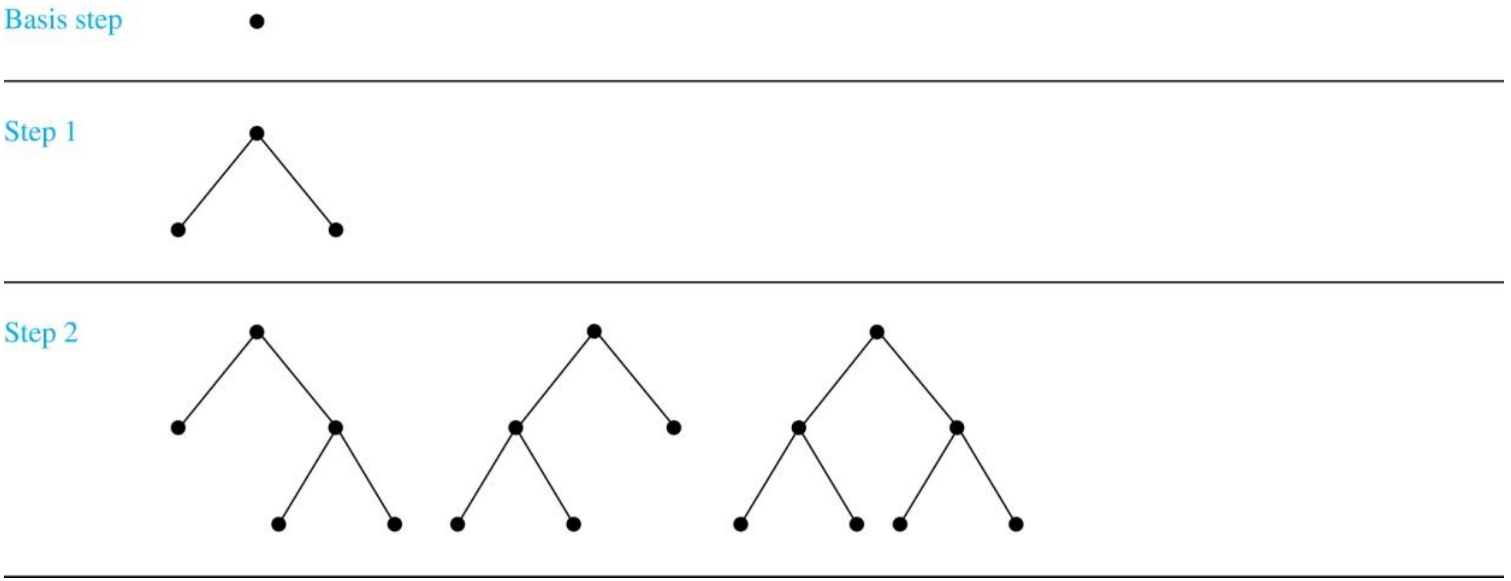
Full Binary Trees

Definition: The set of *full binary trees* can be defined recursively by these steps.

BASIS STEP: There is a full binary tree consisting of only a single vertex *r*.

RECURSIVE STEP: If T_1 and T_2 are disjoint full binary trees, there is a full binary tree, denoted by $T_1 \cdot T_2$, consisting of a root *r* together with edges connecting the root to each of the roots of the left subtree T_1 and the right subtree T_2.

Building Up Full Binary Trees



Structural induction

- To prove results about recursively defined sets we generally use some form of mathematical induction.
- Example: show that the set S defined in example 7 is the set of all positive integers that are multiples of 3.

  Solution: Let *A* be the set of all positive integers divisible by 3. To prove *S*=*A*, we must show that *A* is a subset of *S* and that *S* is a subset of *A*.

  Use mathematical induction

Proof by structural induction

- Basis step: show that the result holds for all elements specified in the basis step of the recursive definition to be in the set.
- Recursive step: show that the statement is true for all elements used to construct new elements in the recursive step of the definition, the result holds for these new elements.

Mathematical induction principle

Recursive definition

$\Rightarrow$ Structural induction

Example

- Show that every well-formed formula for compound propositions, as defined in Example 5, contains an equal number of left and right parentheses.

Proof:

We can use Structural Induction :

Basis Step: Show that the result is true for T, F, and p, whenever p is a propositional variable.

Recursive Step: Show that if the result is true for the compound propositions p and q , it is also true for ($\neg p$), ($p \vee q$), ($p \wedge q$), ($p \rightarrow q$), ($p \leftrightarrow q$).

Example

- Use structural induction to prove that $l(xy)=l(x)+l(y)$, where x and y belong to $\sum^*$, the set of strings over the alphabet $\sum$.

Solution: use definition 2 and example 9

$P(y)$ is the statement that $l(xy)=l(x)+l(y)$

Basis step: $l(x\lambda)=l(x)=l(x)+0= l(x)+ l(\lambda) \Rightarrow P(\lambda)$ is true

Recursive step: prove that $P(y)$ is true, $P(ya)$ is true

$l(xya)=l(xy)+1=l(x)+ l(y)+1= l(x)+ l(ya)$

Examples

- Define the height $h(T)$ of a full binary tree T recursively

- If T is a full binary tree T, then $n(T) \leq 2^{h(T)+1} - 1$

Full Binary Trees

Definition: The *height h(T)* of a full binary tree *T* is defined recursively as follows:

- **BASIS STEP: The height of a full binary tree *T* consisting of only a root *r* is $h(T) = 0$.**
- **RECURSIVE STEP: If T_1 and T_2 are full binary trees, then the full binary tree $T = T_1 \cdot T_2$ has height $h(T) = 1 + \max(h(T_1), h(T_2))$.**

- **The number of vertices $n(T)$ of a full binary tree *T* satisfies the following recursive formula:**
  - **BASIS STEP: The number of vertices of a full binary tree *T* consisting of only a root *r* is $n(T) = 1$.**
  - **RECURSIVE STEP: If T_1 and T_2 are full binary trees, then the full binary tree $T = T_1 \cdot T_2$ has the number of vertices**

$$n(T) = 1 + n(T_1) + n(T_2).$$

Proof

Theorem: If T is a full binary tree, then $n(T) \leq 2^{h(T)+1} - 1$.

Proof: Use structural induction.

- **BASIS STEP: The result holds for a full binary tree consisting only of a root, $n(T) = 1$ and $h(T) = 0$. Hence, $n(T) = 1 \leq 2^{0+1} - 1 = 1$.**
- **RECURSIVE STEP: Assume $n(T_1) \leq 2^{h(T1)+1} - 1$ and also $n(T_2) \leq 2^{h(T2)+1} - 1$ whenever T_1 and T_2 are full binary trees.**

$$
\begin{aligned}
n(T) &= 1 + n(T_1) + n(T_2) && (\textit{by recursive formula of } n(T)) \\
&\leq 1 + (2^{h(T1)+1} - 1) + (2^{h(T2)+1} - 1) && (\textit{by inductive hypothesis}) \\
&\leq 2 \cdot \max(2^{h(T1)+1}, 2^{h(T2)+1}) - 1 \\
&= 2 \cdot 2^{\max(h(T1),h(T2))+1} - 1 && (\max(2^x, 2^y) = 2^{\max(x,y)}) \\
&= 2 \cdot 2^{h(t)} - 1 && (\textit{by recursive definition of } h(T)) \\
&= 2^{h(t)+1} - 1
\end{aligned}
$$

Generalized induction

- We can extend mathematical induction to prove results about other sets that have the well-ordering property besides the set of integers.
- Lexicographic ordering: $(x_1,y_1) \leq (x_2,y_2)$ if either $x_1<x_2$, or $x_1=x_2$ and $y_1<y_2$. Can be extended to k-dimension

Example

Suppose that $a_{m,n}$ is defined recursively for $(m,n) \in N \times N$ by $a_{0,0} = 0$ and

$$a_{m,n} = \begin{cases} a_{m-1,n} + 1 & \text{if } n = 0 \text{ and } m > 0 \\ a_{m,n-1} + n & \text{if } n > 0. \end{cases}$$

Show that $a_{m,n} = m + n(n+1)/2$ for all $(m,n) \in N \times N$, that is for all pairs of nonnegative integers.

Solution :

Basis step : $a_{0,0} = 0$

Recursive step : Suppose that $a_{m',n'} = m' + n'(n'+1)/2$ whenever (m',n') is less than (m,n) in the lexicographic oedering of $N \times N$.

Then prove $a_{m,n} = m + n(n+1)/2$.

Exercises

- Page 378: 24, 30, 37, 38, 46, 47

5.4 Recursive algorithms

- Definition: An algorithm is called recursive if it solves a problem by reducing it to an instance of the same problem with smaller input.

Example

- Give a recursive algorithm for computing a^n where a is a nonzero real number and n is a nonnegative integer.

Solution:

procedure *power*(*a*,*n*)

if n=0 then *power*(*a*,*n*):=1

else *power*(*a*,*n*):=$a \cdot$ *power*(*a*,*n*-1)

Example

- Devise a recursive algorithm for computing b^n **mod** m, where b, n, and m are integers with $m \geq 2$, $n \geq 0$, and $1 \leq b \leq m$.

Solution: $(a \bmod m)(b \bmod m) \bmod m = ab \bmod m$

$$\Rightarrow b^n \bmod m = ((b \bmod m)(b^{n-1} \bmod m)) \bmod m$$

Procedure *mpower*(*b*,*n*,*m*)

if n=0 then

mpower(*b*,*n*,*m*)=1

else if n is even then

mpower(*b*,*n*,*m*)=*mpower*(*b*,*n*/2,*m*)^2 mod m

else

mpower(*b*,*n*,*m*)=(*mpower*(*b*, $\lfloor n/2 \rfloor$, *m*)^2 mod $m \cdot b$ mod m) mod m

Note: *mpower*(*b*,*n*,*m*)=b^n mod m

Example

- Give a recursive algorithm for computing the greatest common divisor of two nonnegative integers a and b with $a < b$.

Solution:

Procedure $gcd(a,b)$

if a=0 then $gcd(a,b) = b$

else $gcd(a,b) = gcd(b \bmod a, a)$

Example

- Construct a recursive version of a binary search algorithm.

  Procedure *binary-search*(*x*,*i*,*j*)

  $m := \lfloor (i+j)/2 \rfloor$

  if $x = a_m$ then

  location:=*m*

  else if ($x < a_m$ and $i < m$) then

  binary-search(*x*,*i*,*m*-1)

  else if ($x > a_m$ and $j > m$) then

  binary-search(*x*,*m*+1,*j*)

  else *location*:=0

Recursion and iteration

- Example: $n!$

- Procedure *factorial*(*n*)
  if n=1 then *factorial*(n):=1
  else *factorial*(n):=$n \cdot$*factorial*(n-1)

- Procedure *iterative-factorial*(*n*)
  x:=1
  for i:=1 to n
    x:=$i \cdot x$
  {x is $n!$}

Computational comparison

- Example: Fibonacci numbers

- Procedure *fibonacci*(*n*)

  if n=0 then *fibonacci*(0) :=0

  else if n=1 then *fibonacci*(1) :=1

  else *fibonacci*(*n*) := *fibonacci*(*n*-1) + *fibonacci*(*n*-2)

Fibonacci numbers

- **Procedure** ***iterative-Fibonacci*(*n*)**

```
if n=0 then y:=0
else
begin
    x:=0
    y:=0
    for i:=1 to n-1
    begin
        z:=x+y
        x:=y
        y:=z
    end
end
  {y is the nth Fibonacci number}
```

Merge sort

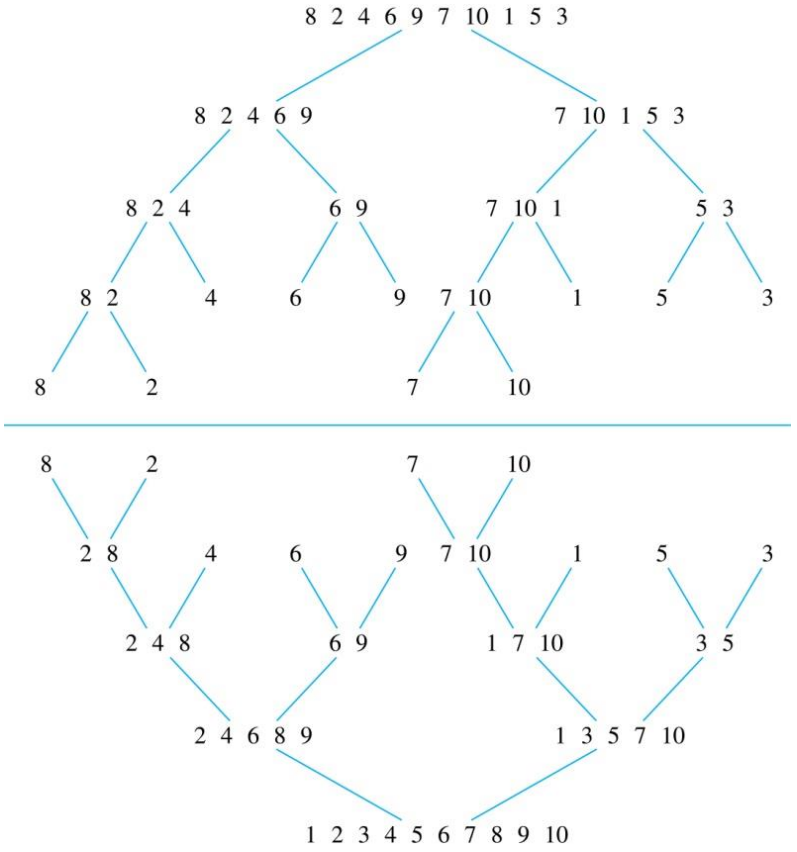
- *Merge Sort* works by iteratively splitting a list (with an even number of elements) into two sublists of equal length until each sublist has one element.
- Each sublist is represented by a balanced binary tree.
- At each step a pair of sublists is successively merged into a list with the elements in increasing order. The process ends when all the sublists have been merged.
- The succession of merged lists is represented by a binary tree.

Example

Use merge sort to put the list

8,2,4,6,9,7,10, 1, 5, 3

into increasing order.



Merge sort

- **Algorithm: a recursive merge sort**

  Procedure ***mergesort***$(L=a_1, \ldots, a_n)$

  if n>1 then

  m:=$\lfloor n/2 \rfloor$

  $L_1 := a_1, a_2, \ldots, a_m$

  $L_2 := a_{m+1}, a_{m+2}, \ldots, a_n$

  $L :=$ ***merge***$($***mergesort***(L_1),***mergesort***$(L_2))$

  {*L* is now sorted into elements in nondecreasing order}

Merging two lists

- **Procedure merge(L_1, L_2)**

  L:=empty list

  while L_1 and L_2 are both empty

  begin

  remove smaller of first element of L_1 and L_2 from the list it is in and put it at the right end of L

  if removal of this element makes one list empty then remove all elements from the other list and append them to L

  end

  {L is the merged list with elements in increasing order}

Theorem

- The number of comparisons needed to merge sort a list with n elements is $O(n\log n)$

Exercises

- Page 391: 12, 45, 50

5.5 Program correctness

- We need a proof to show that the program always gives the correct output.
- Only limited progress has been made toward this goal. Indeed, some mathematicians and theoretical computer scientists argue that it will never be realistic to mechanize the proof of correctness of complex programs.